

Architectural Analysis and System Architecture

SportsHub Enterprise

Architecture decision: Use a layered FastAPI modular monolith for the Sportshub web release, supported by PostgreSQL, Redis, REST/SSE, background workers, and provider adapters. Deploy behind secure cloud edge services, preserve clear module boundaries, and extract only capabilities whose scale or availability requirements justify separation.

Prepared by: Samson Kitigo

Student ID: 618666

Course project: SportsHub Enterprise

Executive Summary

SportsHub Enterprise is a responsive, web-first sports information platform. The Vision document is the authority for product scope: the initial release provides secure accounts, favorites and preferences, live scores, fixtures, standings, player and team statistics, personalized notifications, official ticket discovery, and an administrative portal.

The selected solution is a layered FastAPI modular monolith. Versioned endpoints and Pydantic schemas call domain services and repositories; PostgreSQL is the system of record; Redis accelerates high-read sports data and supports realtime state; background workers ingest provider data and deliver notifications; and stable adapters isolate sports, ticketing, identity, and notification providers. A load-balanced cloud deployment supports growth while retaining the implementation simplicity appropriate for the course project.

Scope rule: The initial architecture includes the responsive web application and admin portal only. Native mobile apps, AI predictions and recommendations, social communities, in-app messaging, merchandise, and premium subscriptions remain explicit future-scope items.

1. Architectural Scope and Context

The initial system serves fans and administrators through a responsive browser interface. Fans create accounts, select favorite teams or competitions, view current and historical sports information, receive personalized alerts, and follow links to official ticket providers. Administrators manage users, competitions, notifications, integrations, platform configuration, and operational analytics.

MVP release	Explicit future releases
Responsive web UI; authentication; favorites and preferences; live scores; fixtures; standings; team/player statistics; notifications; ticket discovery; admin portal.	Native Android/iOS; AI predictions and recommendations; social communities; in-app messaging; merchandise marketplace; premium subscriptions.

1.1 System Boundary

SportsHub owns accounts, roles, favorites, preferences, normalized and cached sports records, notification rules and history, ticket-discovery mappings, administrative configuration, and audit or analytics data. It does not own authoritative sports feeds, ticket inventory, purchase fulfillment, external identity systems, or email, web-push, or SMS networks. Those capabilities remain external and are accessed through controlled adapters.

2. Architectural Drivers

2.1 Functional Drivers

- Secure registration, sign-in, profile management, and role-based fan and administrator access.
- Favorite teams or competitions and stored preferences that personalize views and notifications.

- Live scores, fixtures, standings, and player or team statistics across supported competitions.
- Timely, preference-aware notifications for relevant match and competition events.
- Ticket discovery through official provider links, with purchase and fulfillment outside SportsHub.
- Administrative management of users, competitions, notifications, analytics, system configuration, and integrations.

2.2 Quality Attribute Scenarios

Attribute	Scenario	MVP architectural response
Performance	Most pages load in under two seconds under normal conditions, while live updates arrive with minimal delay.	CDN delivery, Redis caching, precomputed views, concurrent provider calls, SSE, and asynchronous ingestion; validate targets with page and live-path tests.
Scalability	Audience and event volume can grow toward millions of users and high-profile match bursts.	Stateless load-balanced API capacity, distributed cache, separately scalable workers, durable event queues, and indexed PostgreSQL access.
Availability	A provider or instance failure should cause controlled degradation and minimal downtime.	Health checks, multiple instances, cached last-known data, timeouts, idempotent work, database backups, and tested recovery procedures.
Security	Accounts, preferences, administration, and integration secrets must remain protected.	TLS, secure tokens or OAuth, RBAC, validation, rate limits, secret management, encryption, and auditable administration actions.
Maintainability	The course team must change providers and features without destabilizing unrelated functions.	Endpoint-service-repository layers, stable DTOs, provider adapters, migrations, automated tests, observability, and recorded decisions.

2.3 Constraints and Assumptions

- The initial release is a responsive web application and admin portal; native mobile clients are future scope.
- Sports data, ticket discovery, identity, and message delivery depend on external APIs, quotas, licenses, formats, and latency.
- The course-project schedule and team size favor a modular monolith over independently deployed business microservices.
- PostgreSQL is the durable system of record, Redis is the distributed cache and realtime-state store, and the cloud platform must support scaling, monitoring, backup, and recovery.

3. Selected Architecture Style

The selected solution combines four patterns: a layered architecture separating endpoints, services, repositories, and persistence; a modular monolith containing the initial business capabilities in one FastAPI codebase; event-driven and scheduled processing for live sports ingestion and notifications; and adapter-based integration for third-party providers.

It keeps development, testing, deployment, and transactions understandable. Web instances and background workers can scale separately, and a business module should become a microservice only after measured load, reliability, or team-ownership needs justify the operational cost.

3.1 Key Architectural Trade-offs

Decision	Benefit	Cost and architectural response
Modular monolith	Simple releases, transactions, and course-team ownership.	Business modules do not scale independently; preserve boundaries and extract only after measured need.
Redis distributed cache	Fast sports reads and shared realtime state across API instances.	Invalidation and consistency add complexity; use TTLs, versioned keys, ownership rules, and monitoring.
Durable worker queue	Isolates provider latency and notification delivery from web requests.	Adds operational components; require idempotency, retries, dead-letter handling, and queue metrics.
Stable SportsHub DTOs	Protects clients from provider-specific formats and provider replacement.	Requires mapping and contract maintenance; use validation, fixtures, and provider contract tests.

4. High-Level System Architecture

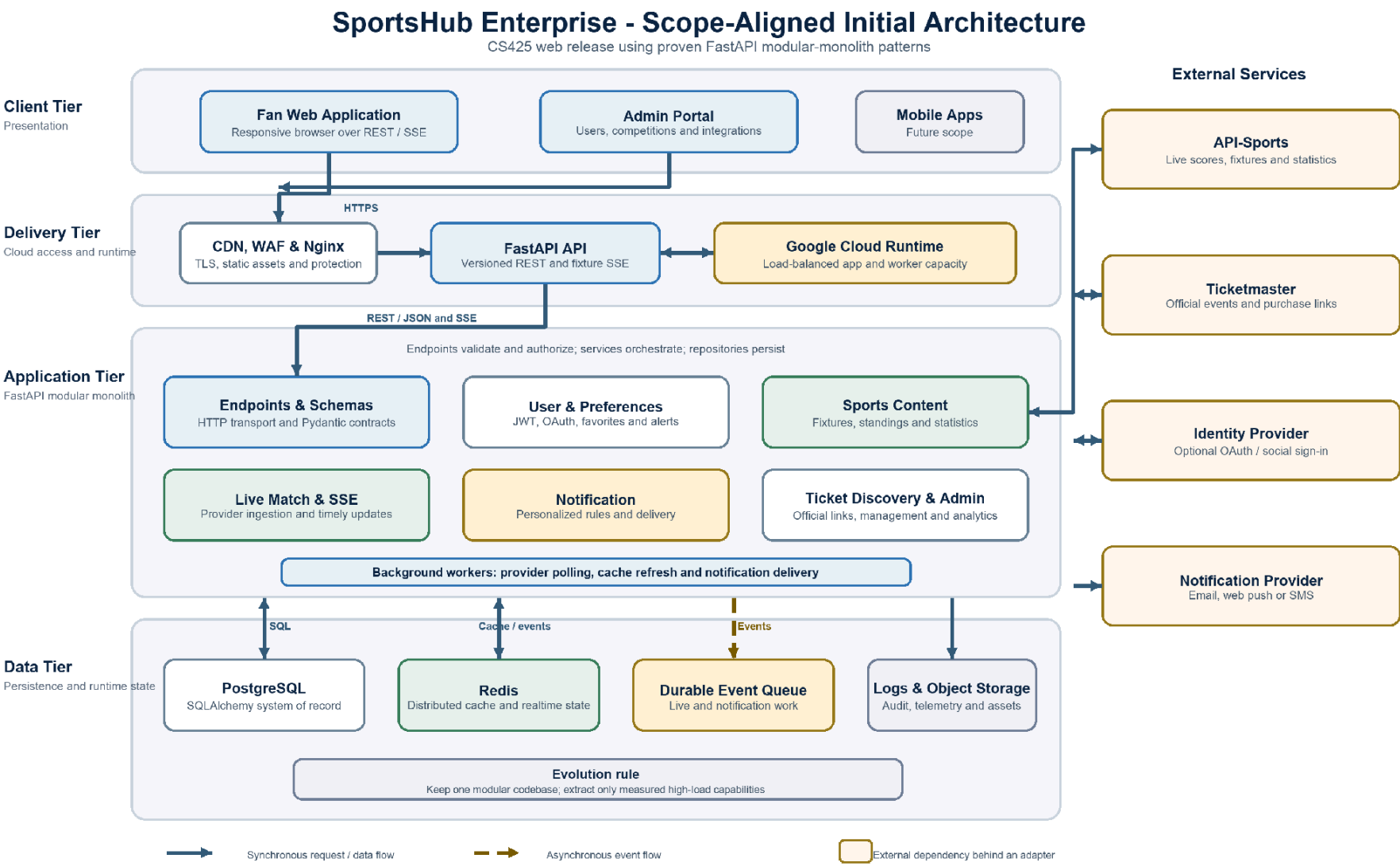


Figure 1. Scope-aligned initial solution architecture for SportsHub Enterprise.

5. Component Responsibilities

Component	Primary responsibility	Key interactions
Fan web application	Delivers the responsive experience for accounts, favorites, sports views, alerts, and tickets.	CDN, REST/SSE API, browser accessibility.
Admin portal	Manages users, competitions, notifications, integrations, configuration, and analytics.	RBAC endpoints, audit logs, PostgreSQL.
Edge and API entry	Terminates TLS, filters traffic, serves static assets, rate-limits, and routes versioned REST/SSE requests.	CDN/WAF/Nginx, FastAPI, fan/admin clients.
Endpoints and schemas	Validate inputs, enforce authorization, define stable contracts, and construct responses.	Services, authentication dependencies, clients.
User and preferences	Handles registration, sign-in, roles, profiles, favorites, and notification preferences.	PostgreSQL, identity provider, notification module.
Sports content	Normalizes fixtures, scores, standings, teams, players, and statistics into SportsHub models.	Sports provider, Redis, PostgreSQL.
Live match and SSE	Polls or receives provider changes, updates current state, and streams timely match events.	Sports provider, Redis, event queue, web clients.
Notifications	Evaluates preferences, records alerts, and delivers through configured channels.	Event queue, PostgreSQL, delivery provider.
Ticket discovery	Matches sports events to official ticket listings and returns provider purchase links.	Ticket provider, sports content; no local payment.
Workers and data services	Run provider ingestion, cache refresh, notification work, persistence, telemetry, and recovery support.	PostgreSQL, Redis, durable queue, logs/storage.

6. Key Runtime Flows

6.1 Fan Access and Personalization

1. The browser loads static assets through the CDN and sends an HTTPS request to a versioned FastAPI endpoint.
2. The endpoint validates the request and applies the required authentication and role dependency.
3. The service resolves favorites and preferences, reads cached sports content where safe, and uses repositories for durable domain data.
4. The endpoint returns stable JSON for the responsive fan or administration view.

6.2 Live Score Update

1. A background worker polls the sports provider or consumes an available provider webhook and validates the payload.
2. The sports service maps provider data into stable SportsHub models, updates cache and durable records, and emits an idempotent event.
3. The live module publishes the update through SSE to relevant browser sessions with reconnect and last-event handling.
4. The notification worker evaluates favorites and preferences, records the notification, calls the configured delivery provider, and tracks delivery outcome.

6.3 Ticket Discovery

A fan opens a fixture and SportsHub resolves its teams, competition, location, date window, and provider identifiers. The ticket adapter searches the configured official provider, filters and ranks matching events, and returns an official purchase link. Payment, ticket inventory, fulfillment, refunds, and customer support remain outside SportsHub.

7. Deployment and Infrastructure

7.1 Target Cloud Deployment

Web assets are delivered through a CDN. A web application firewall and load balancer or Nginx route HTTPS traffic to one or more stateless FastAPI application instances. Separately scalable workers process sports-provider ingestion, cache refresh, and notification delivery through a durable queue. Managed PostgreSQL and Redis services provide persistence and shared cache state, while centralized logs, metrics, backups, and object storage support operations. A small course deployment can begin with one application instance without changing these logical boundaries.

7.2 Data Management

- PostgreSQL and SQLAlchemy provide durable storage and transaction boundaries for users, preferences, normalized sports data, notifications, ticket mappings, configuration, and audits.
- Redis provides distributed cache and short-lived realtime state; process-local caching, if used, remains a disposable optimization.
- A durable event queue decouples live-data ingestion and notification delivery from HTTP requests and supports retry, deduplication, and dead-letter handling.
- Automated backups, point-in-time recovery where available, retention rules, and recovery drills protect durable data and operational evidence.

7.3 External Integration Strategy

Provider-specific logic is kept in adapters so endpoints and repositories do not depend directly on upstream protocols. Sports and ticket adapters apply validation, timeouts, bounded retries, quota awareness, and schema mapping. Identity and notification providers use separate contracts so implementation choices can change without rewriting domain services. Secrets come from runtime configuration or a secret manager, never source code, logs, or URLs.

8. Cross-Cutting Architecture Concerns

8.1 Security

TLS protects all traffic. Secure session tokens or standards-based OAuth support authentication, and RBAC separates fan and administrator capabilities. Input validation, password policy, rate limiting, least-privilege database roles, managed secrets, dependency scanning, audit records, and privacy-aware logging reduce attack surface. Administrative and integration changes require explicit authorization and traceability.

8.2 Performance and Scalability

CDN delivery, Redis caching, indexed queries, pagination, precomputed standings, request coalescing, and concurrent provider calls support the under-two-second page goal. Stateless API instances, shared cache state, durable queues, backpressure, and separately scalable workers handle event bursts. Load tests must verify both ordinary browsing and high-interest live matches.

8.3 Availability and Resilience

Health checks, multiple application instances, rolling deployment, cached last-known sports data, timeouts, circuit breaking, bounded retries, idempotent workers, database backups, and documented rollback protect availability. Provider failure degrades only the dependent function: existing sports content remains readable, delayed live data is labeled, and ticket or notification errors do not block core browsing.

8.4 Observability and Maintainability

Request and actor IDs, structured logs, provider and cache metrics, queue depth, worker heartbeats, database-pool metrics, live-update delay, notification outcomes, and tracing form the operational baseline. OpenAPI contracts, Alembic migrations, automated tests, provider fixtures, and architecture decision records keep changes controlled.

9. Architectural Risks and Mitigations

Risk	Potential effect	Mitigation in this iteration
Provider outage or quota	Live scores, statistics, tickets, or delivery become delayed or unavailable.	Cache last-known data, monitor quotas and freshness, bound retries, isolate adapters, and communicate degraded status.
High-interest match burst	Slow pages, dropped live updates, or database saturation.	CDN and Redis caching, load-balanced stateless APIs, queue backpressure, autoscaling signals, and scenario-based load tests.
Provider schema mismatch	Incorrect teams, times, standings, or ticket matches reach users.	Stable internal DTOs, validation, mapping fixtures, contract tests, reconciliation, and quarantined bad payloads.
Notification or ticket partial failure	Duplicate alerts, missing delivery, or broken provider links.	Idempotency keys, retries with dead letters, delivery status, link validation, and failure isolation from core sports views.
Scope creep into future features	Course schedule and architecture quality are diluted by chat, AI, commerce, or native-app work.	Maintain an initial/future scope register and require a reviewed change decision before adding excluded capabilities.

10. Next Architectural Iteration

1. Confirm the supported sports and competitions and select sports-data, ticket, identity, and notification providers after reviewing licenses, quotas, cost, coverage, and reliability.
2. Define measurable SLOs for the under-two-second page goal, live-update delay, availability, recovery point, and recovery time.
3. Create the deployment, network, environment, scaling, and disaster-recovery views that refine this logical architecture.
4. Create the logical data model for users, favorites, sports entities, events, standings, statistics, notifications, ticket mappings, and audit records.
5. Prototype and load-test the complete live path from provider ingestion through cache, event queue, SSE, and personalized notification delivery.
6. Complete a threat model and define administrator authorization, secrets, audit retention, privacy, backup, rollback, and incident procedures.

11. Scope-to-Architecture Traceability

The following mapping verifies that each initial capability has an explicit architectural home without introducing future-release features.

Initial capability	Primary architectural owner	Supporting elements
Registration and authentication	User and Preferences	Secure tokens/OAuth, RBAC, PostgreSQL, identity adapter.
Favorites and preferences	User and Preferences	PostgreSQL profile data, cached personalization, notification rules.
Scores, fixtures, and standings	Sports Content	Sports adapter, normalized DTOs, Redis, PostgreSQL.
Player and team statistics	Sports Content	Provider mapping, indexed queries, cache, stable API contracts.
Live updates	Live Match and SSE	Background ingestion, Redis state, durable event queue, browser SSE.
Personalized notifications	Notification	Favorites/preferences, durable work, delivery adapter, outcome tracking.
Official ticket discovery	Ticket Discovery	Sports-event matching, ticket adapter, official external purchase link.
Administration and analytics	Admin Portal	RBAC endpoints, configuration, audit logs, metrics, PostgreSQL.

Conclusion

The responsive web application, layered FastAPI modular monolith, PostgreSQL, Redis, durable background work, REST/SSE delivery, and provider adapters support the required initial features. Clear boundaries and measurable evolution points leave room for future mobile, AI, social, messaging, marketplace, and subscription capabilities without prematurely building them.